MiniLang Compiler Visualizer

Report 2 — Implementation, Demonstration and Evaluation

1. How the Implemented Modules Work Together

Report 1 went through each backend module on its own — what it is responsible for, and which classical compiler-design idea it implements. This report assumes that description and instead looks at the same set of modules from the other direction: not what each one does in isolation, but how they are wired together into one running pipeline, and what actually happens, step by step, when a MiniLang program is compiled and then executed.

For reference, the pipeline is: app/lexer.py turns source text into tokens; app/parser.py turns tokens into an abstract syntax tree, using the node classes defined in app/ast_nodes.py; app/semantic.py walks that tree, resolving names and computing types; app/tac.py lowers the checked tree into three-address code and groups it into basic blocks and a control-flow graph; app/optimize.py runs the four optimisation passes over that code; app/codegen.py lowers the optimised code into stack-machine assembly; and app/vm.py executes that assembly. app/literals.py sits underneath three of these — optimisation, three-address code generation, and the virtual machine — as the one shared place that decides how a literal is written and computed. app/main.py is the FastAPI layer that calls all of the above, in this exact order, and app/schemas.py defines the shape of the JSON it sends back to the browser. None of this is new information by itself — it is exactly what Report 1 covers in detail, module by module — but writing it as one paragraph rather than eight separate ones is the point of this section.

What actually keeps these files working as one system, rather than eight separate scripts, is a handful of shared conventions enforced through plain Python objects rather than anything elaborate. Every phase after the parser is handed exactly the object the previous phase produced: the semantic analyser receives the Program node the parser returned; the three-address code generator receives that same node once the semantic analyser has annotated it in place with inferred types; the optimiser receives the raw instruction list and the function boundaries the code generator computed; and the target code generator receives the optimiser's output together with each function's parameter list. There is no intermediate serialisation step between phases internally — JSON only appears once, right at the end, when app/main.py packages the results of all six phases into a single response object and sends it to the browser in one HTTP reply. That single-response design, already introduced in Report 1's architecture

section, is what the POST /compile route does line by line: it calls the lexer, then the parser, then the semantic analyser, then the three-address code generator, then the optimiser, then the code generator, catching a parse failure without aborting the whole request, and copying each phase's output onto the response as it goes, along with how many milliseconds that phase took.

POST /run is a separate route rather than a flag on /compile, because running a program and compiling it are different operations with different costs. It is given exactly the assembly listing Build already produced, and it recovers the address of main by scanning that listing for main's own label, rather than keeping a second, separate piece of state that could quietly drift out of sync with the assembly it is supposed to belong to. The same wiring decides what happens when something goes wrong partway through, which section 3 of this report covers in more depth: a lexical mistake does not stop the pipeline at all, because the scanner already recovers and keeps producing a complete token stream; a syntax error does stop it, because there is no tree for the later phases to work from, although the tokens scanned up to that point are still returned so the Lexical tab still has something to show; and a type error stops neither the three-address code generator nor anything after it, because every expression node carries a type by the time semantic analysis finishes with it — correctly inferred, or a placeholder where it could not be — so lowering to three-address code never has to guess.

2. Complete Input-to-Output Demonstration

To make the pipeline concrete rather than abstract, this section follows one single MiniLang program — the project's canonical demo program, kept in the repository at tests/programs/demo.ml — all the way from source text to the two numbers it finally prints. Every number quoted below was taken by actually running this exact program through the real compiler while this report was being written, not estimated by hand.

```
func fib(int n): int {
    int a = 0;
    int b = 1;
    int i = 0;
    while (i < n) {
        int t = a + b;
        a = b;
        b = t;
        i = i + 1;
    }
    return a;
}

func main(): void {
```

```
    int limit = 5 * 2;          // folds to 10
    int unused = 42;            // eliminated by DCE
    int x = (3 + 4) * (3 + 4); // CSE target
    print(fib(limit));
    print(x);
}
```

The program declares two functions. fib computes the n-th Fibonacci number iteratively, using a loop rather than recursion, so the control-flow graph has a real back edge to show. main calls it once with a folded constant, and separately builds a second expression purely to exercise constant folding and common-subexpression elimination, and declares one variable it never reads at all, purely to give dead-code elimination something real to remove. A single twenty-line program therefore still gives every phase, and every optimisation pass, something genuine to do.

[Insert Source tab screenshot here]

2.1 Lexical Analysis

Scanning this file produces 106 tokens in total, each carrying its type, its exact text, and the line and column at which it starts. The first few are func at (1, 1), the identifier fib at (1, 6), an opening parenthesis at (1, 9), the keyword int at (1, 10), the identifier n at (1, 14), a closing parenthesis at (1, 15), a colon at (1, 16), and a second int at (1, 18) — already enough to see the scanner correctly separating the keyword int from the identifier n it introduces, exactly the maximal-munch behaviour described in Report 1.

[Insert Lexical tab screenshot here]

2.2 Syntax and Semantic Analysis

Parsing succeeds without error and produces two top-level function declarations, fib and main, each the root of its own subtree built exactly as the grammar in Report 1 describes. Semantic analysis then resolves every name in the program and records ten symbols in total: the two functions themselves; fib's parameter n and its three local variables a, b and i, all at scope depth 1; the loop body's own local t, one scope level deeper, inside the while block; and main's three locals, limit, unused and x. Every one of the ten is correctly typed int, and no type error is reported — there is nothing in this program that should produce one.

[Insert Syntax tab screenshot here]

[Insert Semantic tab screenshot here]

2.3 Intermediate Code, Basic Blocks and the Control-Flow Graph

Lowering the checked tree into three-address code produces 30 instructions across 6 basic blocks. fib's instructions run from index 0 to 16 and include the loop condition, the body, and the back edge; main's run from index 17 to 29. A short excerpt from main, before any optimisation has touched it, reads:

```
17  main:
18  t4 = 5 * 2
19  limit = t4
20  unused = 42
21  t5 = 3 + 4
22  t6 = 3 + 4
23  t7 = t5 * t6
24  x = t7
25  param limit
26  t8 = call fib, 1
27  print t8
28  print x
29  return
```

Every multiplication, every addition and every assignment here is its own instruction, each with at most two operands and one result, exactly the flat three-address form Report 1 describes. The six basic blocks come out of the leader algorithm applied to this listing together with fib's: fib's loop condition, its body, and the code after the loop are each their own block, and main is simple enough to form one block on its own.

[Insert ICG tab screenshot here]

2.4 Optimisation

With all four passes switched on, the 30 instructions above become 23 — seven removed outright. Constant folding fires four times, constant propagation five times, and dead-code elimination removes seven instructions once liveness analysis confirms none of them is read again. fib's half of the listing is untouched, because nothing in a loop driven by a runtime parameter n is foldable at compile time; the entire effect lands on main, which collapses to this:

```
17  main:
18  param 10
19  t8 = call fib, 1
20  print t8
21  print 49
22  return
```

Reading the two listings side by side shows each pass's fingerprint. 5 * 2 is folded to 10 at compile time, and that 10 is then propagated all the way through limit and into the call itself, so the final listing never mentions limit at all — it pushes the literal 10 directly. unused is declared, assigned 42, and never read again anywhere in the program, so dead-code elimination removes both the assignment and the variable without a trace. (3 + 4) * (3 + 4) folds to 49 through constant folding and propagation alone, without common-subexpression elimination ever needing to act, because each 3 + 4 is already foldable on its own before the multiplication is even reached — a case worth noting precisely because it shows one pass can make another pass's job disappear rather than fire.

[Insert Optimisation tab screenshot here]

2.5 Target Code and Execution

The optimised listing lowers to 46 stack-machine instructions. The tail of main's assembly shows the same collapse the optimiser produced, now expressed as opcodes rather than three-address lines — the call pushes a literal 10 directly, and the second print pushes a literal 49 directly, with no load of any named variable for either:

```
36  main:
37  ENTER 1
38  PUSH 10
39  CALL fib
40  STORE t8
41  LOAD t8
42  PRINT
43  PUSH 49
44  PRINT
45  RET 0
```

Running this assembly on the stack-based virtual machine retires 268 instructions in total, halts cleanly with no runtime error, and prints exactly two lines: 55, then 49. 55 is fib(10) — ten iterations of the loop starting from a = 0, b = 1 produce the tenth Fibonacci number by construction, and the virtual machine's output matches that by hand-computed value exactly, which is the final, end-to-end confirmation that every phase between the source file and this output agrees with every other one.

[Insert Target tab, with Run output visible screenshot here]

3. Test Results, Error Handling and Known Limitations

3.1 Test Results

The project is tested with pytest rather than by hand-checking the browser after every change. As of this report, the suite collects and passes 1016 tests with zero failures, spread across fourteen test files — one for almost every module in app/: test_lexer.py, test_parser.py, test_ast_nodes.py, test_semantic.py, test_tac.py, test_optimize.py, test_codegen.py, test_vm.py, test_literals.py, test_schemas.py, test_logging_config.py, test_main.py, test_samples.py and test_frontend.py — plus a shared conftest.py and the canonical program at tests/programs/demo.ml, which several of these files compile and execute end to end rather than only unit-testing one phase in isolation. The same suite, together with the ruff linter and the black formatter, runs automatically on every push through the GitHub Actions workflow defined in .github/workflows/ci.yml, on both Python 3.10 and Python 3.12, which also starts the FastAPI server itself and checks that it really does produce tokens for a sample program — worth calling out on its own, because a green unit-test run does not by itself prove the application starts.

3.2 Error Handling, Phase by Phase

Each phase is deliberately inconsistent about whether it stops the pipeline, and that inconsistency is a design decision rather than an oversight. The lexer recovers from a small set of mistakes on its own — meeting a lone & it assumes && was intended, reports the mistake with a suggested repair and a confidence score, and keeps scanning rather than giving up, so later phases still have a complete token stream to work with. The parser, by contrast, stops at the first syntax error it meets and raises an exception rather than attempting to recover; this is a deliberate baseline rather than a limitation that was never addressed, kept this way on purpose because a future error-recovery layer, described as future work in section 4, needs exactly this baseline to be measured against. Semantic analysis does not stop at a type error either — it keeps resolving the rest of the program, still returns the full symbol table it has built, and reports every type diagnostic it found rather than only the first one. The virtual machine raises a clearly worded runtime error for the handful of things that are only knowable once a program is actually running rather than at compile time: division or modulo by a literal zero, a stack underflow, a jump to an address that was never resolved, and an invalid program counter — each message names the exact instruction index and the instruction's own text, so the failure is traceable rather than a bare crash.

3.3 Known Limitations

Stated plainly, so that what this project does not yet do is as clear as what it does:

- MiniLang itself has no arrays, structs or generic types, and no standard library beyond a single print statement — the type system is deliberately limited to int, float, bool and void.

- The target is a small stack machine of our own design, not a real instruction set such as x86 or ARM, and there is no register allocation — comparing its performance to a production compiler's output would not be a fair comparison, because that was never the goal.

- The parser does not yet recover from a syntax error and continue. It reports the first one and stops, so a program with three separate mistakes currently needs three separate compiles, each fixing one, before all three are visible. Panic-mode recovery and a ranked suggestion engine are designed into the architecture described in Report 1, but are not implemented in app/ as of this report.

- The Parser Theory Lab tab — FIRST and FOLLOW sets, an LL(1) table, LR(0) item sets, and SLR and LALR table construction with conflict detection — is part of the planned architecture and the POST /analyze-grammar route already exists as a stub, but app/grammar.py and app/table_parser.py have not been written yet, so the tab has nothing behind it.

- The recovery benchmark described in the project plan — thirty seeded buggy programs compared against a baseline parser, with precision and recall reported — has not been run, because it depends on the recovery layer above existing first.

- Testing has exercised many small and medium MiniLang programs through the unit and integration test suite, but no single very large program has been used to stress-test the pipeline's performance at scale.

4. Conclusion and Future Improvements

What exists today is a complete, working compiler for a real, if small, programming language: six phases, each one a genuine implementation of an algorithm a compiler-design course teaches rather than a simplification of it, wired into one pipeline that a browser can watch happen phase by phase, backed by over a thousand automated tests and continuous integration that checks the application actually starts, not only that its functions return the right values in isolation. The demonstration in section 2 of this report is not a cherry-picked example; it is the project's own canonical program, and the numbers quoted there — 106 tokens, 10 symbols, 30 instructions before optimisation and 23 after, 46 assembly instructions, and a final output of 55 then 49 — are reproducible by anyone who clones the repository and runs the same program through it.

The clearest next steps are the ones already scoped out as future work rather than discovered along the way. In the nearer term: building the Parser Theory Lab (FIRST and FOLLOW sets, LL(1), LR(0), SLR and LALR table construction with conflict detection) behind the existing /analyze-grammar stub; the panic-mode error-recovery layer with its ranked suggestion engine, and the thirty-program benchmark that was designed to evaluate it against a baseline parser; and a step-through parse animation driven by the table-driven parser's own trace, which the project plan identifies as the single most visually convincing addition available. Further out, and explicitly out of scope for an educational compiler of this kind rather than simply unfinished: real register allocation, loop-invariant code motion and strength reduction, and a target architecture closer to something a real machine executes. A smaller, low-effort extension already partly scoped in the project plan — parameterising the existing lexer by language so the Lexical tab can also tokenise C, C++ and Java source, without attempting to compile any of them — remains available as well, and would demonstrate what is and is not language-specific about lexical analysis on its own.

5. GitHub Repository and Source Code

The complete source code for this project — every module described in both reports, the test suite, the frontend, and the report-generation scripts used to produce these two documents themselves — is kept in a single public repository. The links are given here exactly as they appear in the project's own git configuration, so that they can be copied without transcription errors:

```
Repository (browse):  https://github.com/hardikahujaa/MiniLang
Clone (HTTPS):        https://github.com/hardikahujaa/MiniLang.git
License:              MIT  (see LICENSE in the repository root)
Continuous Integration: .github/workflows/ci.yml
```

The repository's top level is organised the same way this report describes it: app/ holds the nine compiler and server modules covered in section 1; static/ holds the frontend (index.html, app.js, viz.js and style.css); tests/ holds the fourteen test files and the canonical demo program referred to throughout section 2 and section 3; and report/ holds the scripts that generated this document and its companion. requirements.txt and requirements-dev.txt list the runtime and development dependencies respectively, run.bat is the one-click Windows launcher described in section 6, and pyproject.toml holds the shared configuration for black and ruff that the continuous integration workflow enforces on every push. This repository is the single source of truth: every

grammar rule, every algorithm, and every number quoted anywhere in either report can be reproduced directly from what is checked in there.

6. Installation and Workflow Guide

6.1 Getting the Code

```
git clone https://github.com/hardikahujaa/MiniLang.git
cd MiniLang
```

6.2 Running It on Windows (recommended)

Double-click run.bat in the repository root, or run it from a terminal. It creates a virtual environment the first time it is run if one does not already exist, installs the project's dependencies into it, picks a free port (8000, or 8010 if that one is already taken), opens the default browser to the running application, and starts the server. This is the launcher the project itself uses, written specifically to avoid a confusing error that comes from this machine having more than one Python installation on PATH, none of which is necessarily the one with the project's dependencies installed.

6.3 Running It Manually (any operating system)

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# macOS / Linux:
source .venv/bin/activate

pip install -r requirements.txt
uvicorn app.main:app --reload
```

Once the server is running, open http://127.0.0.1:8000 in a browser. No separate frontend build step is needed — static/index.html, app.js and viz.js are served directly, and D3.js is vendored into the repository so the application works with no internet connection once installed.

6.4 Running the Test Suite

```
pip install -r requirements-dev.txt
pytest -q
```

requirements-dev.txt already pulls in requirements.txt, so this one install line is enough on a fresh checkout. This runs the same 1016 tests described in section 3 and should finish in well under ten seconds.

6.5 Using the Application

The workflow the UI is built around is: write or paste a MiniLang program into the editor on the Source tab; press Build; and then walk through the remaining tabs in order — Lexical for the token stream, Syntax for the parse tree, Semantic for the symbol table and any type errors, ICG for the three-address code and control-flow graph, Optimisation to toggle each of the four passes on and off and watch the instruction count change, and Target for the final assembly. Pressing Run on the Target tab executes whatever assembly the last Build produced and shows the program's actual output. Editing the source and pressing Run again without rebuilding first still runs the previous build, matching the button's own disabled state until Build is pressed again — editing only takes effect once Build has been pressed.

7. Evaluation Rubric — Review 2 and Review 3

Both Review 2 and Review 3 for this project are evaluated against the same split: explanation and live demonstration of the working system account for 70 percent of the mark, and the written report accounts for the remaining 30 percent. In practice, this means the larger share of preparation time is best spent on being able to walk an examiner through the running application confidently — opening the Source tab, building a program, narrating what each subsequent tab shows and why it shows that, and running the compiled result to a real answer, the way section 2 of this report does on paper. The two reports (this one and the design-and-architecture report that precedes it) exist to satisfy that remaining 30 percent and to give an examiner a written record to read independently of the demo, but neither one replaces being able to explain, out loud and from memory, what the code in front of the examiner is actually doing at the moment it is asked about.